

Categorizing the Long Tail: An Empirical Study of Web-Search-Augmented Fallback for Embedding-Based Transaction Classification

Surya Parida
Independent Researcher
`surya@finlens.app`

September 2026

Abstract

Embedding retrieval over a merchant index is the workhorse of production transaction categorization: cheap, fast, and accurate on merchants the index has seen. We show on five years of real government purchase-card transactions that its failure is not graceful. Under a strictly temporal split, top-1 accuracy is 0.93 to 0.98 for merchants seen at least once during training and 0.60 for merchants never seen, and the never-seen group is 19% of test volume. The cliff sits at zero prior observations, not at low frequency: a merchant seen exactly once is classified as well as one seen a thousand times. This makes the problem an evidence problem rather than a sample-size problem, so we route only the low-confidence retrievals to a language model that is given web search results for the merchant string. The routed system lifts tail macro-F1 from 0.607 to 0.733 and overall macro-F1 from 0.807 to 0.861 while sending 10.4% of transactions to the model, at \$0.54 per thousand transactions. The web evidence, not the language model, does the work: with prompts, merchants, and taxonomy held fixed, adding search results is worth +0.066 to +0.175 accuracy across five models, including three open-weight ones, and every paired-bootstrap interval excludes zero. On 800 merchants from a second government dataset that never appear in the training index, retrieval scores 0.392 and the same fallback scores 0.576. Writing resolved merchants back into the index over a 70,333-transaction stream halves the fallback bill for 0.3 points of macro-F1, but 47% of the written labels are wrong, which is a warning about self-training on model output. A manual taxonomy of 200 residual errors finds that two thirds are annotation or taxonomy artifacts rather than model mistakes, which bounds how much of the remaining gap is worth chasing. All API responses are cached and committed; a single command regenerates every table and figure in this paper with no network access.

1 Introduction

Every bank, card issuer, and personal-finance application must turn a raw acquirer descriptor such as `SQ *BLUE BOTTLE 1234` or `WW GRAINGER 912` into a spending category. The dominant production recipe is retrieval: normalize the descriptor, embed it with a sentence encoder, search an index of merchants whose categories are already known, and copy the nearest neighbor’s label. It is attractive because it is cheap, it runs in under a millisecond per transaction on a CPU, and it is trivially updated by adding rows to the index.

It also has a known weak point, and the first contribution of this paper is to measure how weak. Merchant frequency is close to a power law: in our training window the rank-frequency

slope is -1.10 and 47% of merchants appear exactly once. Practitioners therefore speak of a “long tail” problem and assume that accuracy degrades smoothly as frequency falls. It does not. Under a temporal split on real card data, retrieval accuracy is 0.96 for merchants seen once, 0.95 for merchants seen two to five times, and 0.98 for merchants seen six to twenty times. It is 0.60 for merchants never seen before, and those merchants carry 19% of test transactions. The distribution is bimodal, not sloped: either the index contains the merchant string and a nearest-neighbor lookup is near-perfect, or it does not and the encoder is reduced to guessing from surface form.

That reframing determines the fix. If the tail were a sample-size problem, more training data or a better encoder would close it. If it is a missing-knowledge problem, then the system needs a source of facts about merchants outside the index, and the cheapest such source is a web search. We therefore study a routed architecture: retrieval handles everything it is confident about, a similarity gate detects the rest, and those transactions alone are sent to a language model that is shown web search results for the merchant string.

We ask four questions, and answer each with a controlled experiment on real data:

1. **Does the routed system beat retrieval, and at what cost?** Tail macro-F1 rises from 0.607 to 0.733 and overall macro-F1 from 0.807 to 0.861, with 10.4% of transactions routed to the model, for \$0.54 per thousand transactions (Section 7).
2. **Is the gain from the web evidence or from the language model?** We hold the prompt, merchant list, and taxonomy fixed and toggle only the search results. Web evidence is worth +0.124 accuracy for gpt-5-mini and +0.175 for gpt-5-nano, and +0.066 to +0.140 for three open-weight models served on a free endpoint. Every 95% paired-bootstrap interval excludes zero (Section 8). Without evidence, the largest model we test is a worse categorizer than a 384-dimensional nearest-neighbor lookup.
3. **Does it transfer to merchants from a different institution?** On 800 Oklahoma purchase-card merchants that never appear in the Washington DC index, retrieval scores 0.392 macro-F1 and the web-augmented fallback scores 0.576 (Section 8.1).
4. **Can the system heal itself?** Writing resolved merchants back into the index over a stream cuts the fallback rate from 2.99% to 1.37% and the bill from \$22.62 to \$10.38, for 0.3 points of macro-F1. But 47% of the written labels are wrong, and a confidence gate on the write path removes the cheap wins along with the errors (Section 9).

Two further results are, we think, the most useful for practitioners. First, the accuracy ceiling is not where the metrics suggest: a hand-labeled taxonomy of 200 residual errors attributes 50% to taxonomy mismatch, where the model’s answer is defensible and the merchant-category-code label is merely different, and 17% to outright label noise, against 8% genuine model reasoning errors (Section 11). Second, the expensive knobs do not matter much: a 1536-dimensional commercial embedding buys one point of tail macro-F1 over a 384-dimensional CPU model at $23\times$ the encoding latency (Section 10), and letting the model run its own searches instead of reading five fixed snippets costs $13\times$ more for 0.7 points (Section 8.2).

Everything in this paper is reproducible without network access. Every API response, 18,928 language model responses and 3,869 searches, is content-hash cached and committed to the repository alongside the frozen prompts, pinned model identifiers, and embedding caches; `reproduce.py` replays them, regenerates every result table, and verifies each one against the committed copy. Total cash spend for the entire study was \$9.45.

2 Related work

Transaction categorization. Published work on categorizing card transactions is mostly industrial and mostly on proprietary data [7]. Descriptors are short, abbreviated, processor-mangled strings, which puts the task closer to entity resolution than to topic classification. Our contribution is not a new classifier but a measurement: the shape of the failure under a temporal split, and what a web-search fallback is and is not worth against it, on data anyone can download.

Retrieval and dense encoders. We use sentence encoders [12–14, 16] over an HNSW index [9], the standard production stack. FinBERT [1] appears as a domain-pretrained control.

Retrieval-augmented generation. Supplying a language model with retrieved text at inference time is standard practice [8], as is letting a model issue its own searches [10]. Our setting is unusual in that the retrieved text is about a named entity the index has never seen, and in that we can measure the value of the evidence separately from the value of the model by toggling only the evidence.

Selective prediction and cascades. Routing low-confidence inputs to a more expensive model is selective prediction [4, 6] combined with a model cascade [2]. Our gate is deliberately the simplest possible: a threshold on top-1 cosine similarity, which we show is a strong signal in this domain (Section 6.1).

Label noise. Benchmark labels are noisier than reported accuracies imply [11]. We audit 300 labels by hand and report both the resulting noise rate and a failure taxonomy, because in this domain the label noise is comparable in size to the remaining model error.

3 Data

Primary dataset. Washington DC publishes every purchase-card transaction made by city agencies, with the acquirer descriptor, the transaction date, and a merchant category code description. The descriptors are real, including processor prefixes, store numbers, and truncation. We split strictly by time: transactions through 2023-12-31 build the index, transactions from 2024-01-01 onward are the test set (Table 1). A random split would leak tail merchants across the boundary and erase the phenomenon this paper is about.

Second dataset. Oklahoma publishes an equivalent state purchase-card file. We use it only for cross-institution cold start: 800 merchants that appear in Oklahoma and never in the DC training window, classified against the DC index and the DC taxonomy.

Controlled instrument. To vary tail severity on demand, which no fixed real dataset allows, we generate synthetic statements from 2,663 real US brand names taken from the OpenStreetMap name-suggestion index, passed through a templated noise model (processor prefixes, store numbers, truncation) with Zipf-distributed frequencies. Two dials are swept: the Zipf exponent and the share of brands that appear only after the split date. The generator is used only for the sweeps in Section 6; no headline number depends on it.

Table 1: Primary dataset and temporal split. The tail is a quarter of test volume, and one test transaction in five comes from a merchant the index has never seen.

Washington DC Purchase Card	Value
Training window	2019-01-01 to 2023-12-31
Test window	2024-01-01 onward
Training transactions / merchants	189,581 / 17,166
Test transactions / merchants	90,466 / 10,443
Tail merchants (train freq ≤ 3)	8,093
Tail share of test transactions	25.3%
sensitivity: freq ≤ 2 / ≤ 5	23.7% / 27.8%
Test transactions from never-seen merchants	19.1%
Rows without a usable MCC	0.17%
Rows with an ambiguous MCC (excluded)	8.9%
Label noise (300-row audit)	4.4% wrong, 19.7% upper bound

Labels. Merchant category codes are mapped to 15 macro categories through a committed CSV built from inspectable range rules plus hand-written overrides. Four “not elsewhere classified” codes (5999, 5399, 7399, 8999) are flagged as ambiguous and excluded from every table; they are 8.9% of rows. To quantify label quality we hand-judged a random sample of 300 labeled transactions: 4.4% are clearly wrong and a further 16% are unclear from the descriptor alone, giving a noise rate between 4.4% and 19.7%. This matters for interpreting Section 11.

Privacy. Only the normalized merchant string ever leaves the machine. The loaders drop card-holder names, amounts, item descriptions, and agency at the boundary, and the search and language model clients accept a merchant string only. Raw and processed data are never committed; a download script fetches them from the two open-data portals.

4 System

The pipeline has five stages, each deliberately simple so that the comparison is about evidence rather than engineering.

Normalization. A conservative rule set uppercases the descriptor, strips known processor and channel prefixes (SQ *, TST*, PAYPAL *, POS PURCHASE, and others), removes store numbers, dates, trailing top-level domains, and digit-heavy reference tokens, and collapses whitespace. It never maps one brand to another, and every applied rule is recorded for audit.

Index. Unique training merchants are embedded with a sentence encoder (all-MiniLM-L6-v2 unless stated) and inserted into an HNSW index ($M = 32$, $ef_{\text{construction}} = 200$, $ef_{\text{search}} = 64$) under cosine distance, carrying the majority category, training frequency, and first-seen date. Inserts are single-threaded so that a seed fixes the graph exactly.

Gate. A query returns its top $k = 5$ neighbors; the prediction is the top-1 label. The gate routes a transaction to the fallback when the top-1 cosine similarity falls below $t = 0.65$.

Fallback. The merchant string is sent to a search API (Firecrawl, five results), and the titles, URLs, and snippets are pasted into a frozen prompt that also lists the 15 categories and asks for JSON with a category, a confidence, and a one-sentence justification. The no-evidence condition uses an identical prompt with the evidence block removed. Prompts are frozen by SHA-256 after a 100-merchant pilot, and the cache key includes the prompt hash, so editing a prompt creates new cache entries rather than silently reusing old ones.

Write-back (optional). In the streaming experiment, a resolved merchant may be inserted into the index with the model’s label so that later transactions from the same merchant are answered by retrieval.

Cost model. Costs are list prices at the time of collection, counted per call from recorded token usage, plus \$0.0107 per search. Open-weight models run on NVIDIA’s free endpoint; their dollar figures are reference rates for equivalently sized hosted models and are marked as such.

5 Experimental setup

All headline numbers are macro-F1 over the union of gold and predicted labels, averaged over five seeds (42–46), with 95% confidence intervals from bootstrap resampling [3] over seeds, and Wilson intervals [15] for binomial rates. Seeds vary the index construction and the evaluation sample, not the cached model responses, which are deterministic replays.

Because the tail is only a quarter of transactions, evaluating every API-backed method on the full test set would spend most of the budget re-confirming head accuracy. We therefore draw a frozen *fallback evaluation set* of 3,000 merchants, 2,500 tail and 500 head, one row per merchant, and evaluate every method on exactly those strings. Merchant-level macro-F1 on this set is the metric in Table 4; it is deliberately harder than the transaction-weighted number, since frequent merchants are not counted repeatedly. Transaction-level results on the full test set appear in Sections 9 and 10.

The three open-weight models are served by a free endpoint that rate-limits to 40 requests per minute and serializes concurrent calls, so they are evaluated on a shared 300-merchant tail subset rather than the full set. Their intervals are correspondingly wider, and we never compare them to the commercial models on different subsets.

6 The shape of the tail

Merchant frequency follows the expected power law (Figure 1, left). The interesting result is what that does to accuracy.

Table 2 breaks kNN accuracy down by how often the test merchant appeared during training. Accuracy is flat and high, between 0.93 and 0.98, for every bin with at least one prior observation, including the singleton bin. It collapses to 0.599 for merchants with zero prior observations. In other words, the “long tail problem” in this domain is not a gradient; it is a membership test. Once a merchant string is in the index, one occurrence is as good as a thousand, because the match is essentially exact. The gap between 0.96 at frequency one and 0.60 at frequency zero is the entire problem.

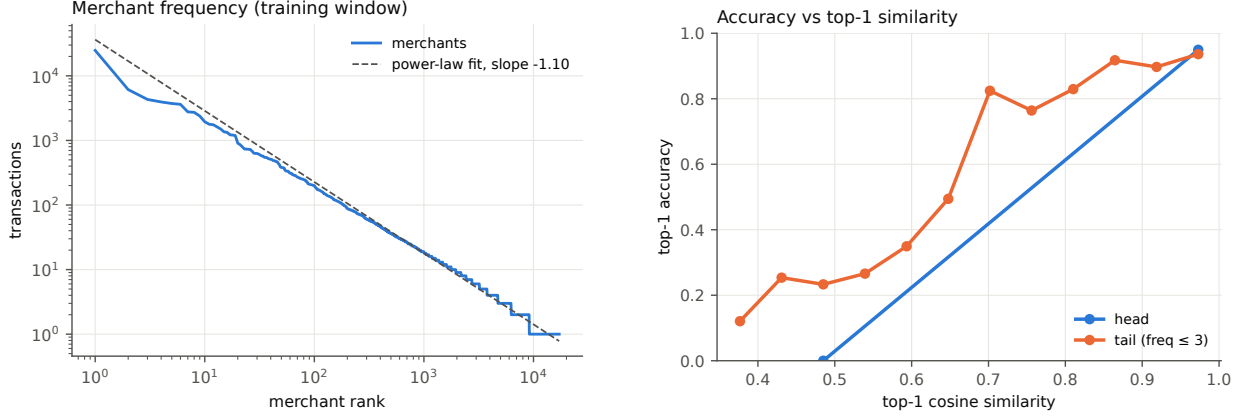


Figure 1: Left: merchant rank-frequency in the training window, log-log, with a fitted slope of -1.10 ; 47% of merchants occur exactly once. Right: top-1 accuracy against top-1 cosine similarity, head and tail. The two panels together are the argument for the routed design: the tail is large, and the encoder knows when it is lost.

Table 2: kNN accuracy by training frequency (MiniLM-L6-v2, full DC test set). The cliff is at zero: a merchant seen once is classified as well as a merchant seen a thousand times.

Training frequency	Test txns	Top-1 accuracy	95% CI
0 (unseen)	14,983	0.599	[0.591, 0.607]
1	2,168	0.960	[0.951, 0.968]
2-5	4,793	0.950	[0.944, 0.956]
6-20	13,594	0.984	[0.981, 0.986]
21-100	9,228	0.970	[0.966, 0.973]
100+	38,349	0.931	[0.928, 0.933]

The generator confirms the mechanism. Sweeping the Zipf exponent from 0.8 to 1.5 changes the tail’s share of traffic from 60% to 28% but moves tail accuracy only from 0.904 to 0.939. Sweeping the share of brands that appear only after the split date has a far larger effect (Table 3): overall accuracy falls from 0.960 to 0.669 as novel brands go from 0% to 30%, and accuracy on genuinely unseen entities sits near 0.5 no matter which dial is turned. Skew is not what hurts; novelty is.

6.1 The gate

For routing to be economical, the encoder must know when it is lost. It does. Top-1 cosine similarity separates the two regimes cleanly (Figure 1, right): below the 0.65 threshold, accuracy is 0.325; at or above it, 0.934. Only 7.9% of test transactions fall below the threshold, and 98.4% of those are tail merchants. A threshold on a single scalar is therefore enough to buy most of what a learned gate could offer, which is why we keep it.

7 Main comparison

Table 4 compares retrieval, language-model-only, and routed systems on the 3,000 merchant evaluation set.

Table 3: Generator sweep over the share of brands that appear only after the split date, with the Zipf exponent held at 1.1 (five seeds). Accuracy on genuinely unseen entities is pinned near 0.5 regardless of the dial; only their share of traffic changes.

Novel brand share	Unseen txns	Overall	Tail	Unseen entities	Seen entities
0.00	1.1%	0.960	0.912	0.474	0.966
0.05	12.1%	0.918	0.841	0.540	0.967
0.10	22.1%	0.867	0.765	0.513	0.966
0.20	39.7%	0.770	0.652	0.473	0.968
0.30	58.8%	0.669	0.570	0.459	0.968

Table 4: Main comparison on the 3,000-merchant fallback evaluation set (2,500 tail, 500 head), macro-F1, five seeds. Cost is US dollars per 1,000 transactions at list prices; fallback is the share of transactions sent to the LLM.

Method	Overall	Head	Tail	USD/1k	Fallback
<i>Retrieval only</i>					
kNN, MiniLM-L6-v2	0.807	0.952	0.607	–	–
kNN, bge-small-en-v1.5	0.820	0.952	0.637	–	–
kNN, text-embedding-3-small	0.822	0.952	0.643	–	–
Logistic regression on MiniLM embeddings	0.619	0.749	0.399	–	–
<i>LLM only, every merchant</i>					
gpt-5-nano, no web	0.444	0.461	0.354	<0.01	100.0%
gpt-5-nano, web evidence	0.542	0.541	0.485	1.59	100.0%
gpt-5-mini, no web	0.507	0.518	0.424	0.03	100.0%
gpt-5-mini, web evidence	0.588	0.576	0.543	1.63	100.0%
<i>Routed: kNN, similarity gate at 0.65, LLM with web evidence</i>					
Routed, gpt-5-nano	0.799	0.952	0.670	0.53	10.4%
Routed, gpt-5-mini	0.861	0.952	0.733	0.54	10.4%

Open-weight models are evaluated on a 300-merchant tail subset and appear in Table 5. The routed system is the only configuration that keeps head accuracy while moving the tail.

Three things stand out. First, **the language model alone is a bad categorizer**: gpt-5-mini with web evidence reaches 0.588 macro-F1 against 0.807 for a 22M-parameter encoder doing nearest-neighbor lookup, and 0.507 without evidence. Whatever the fallback is for, it is not replacing retrieval. Second, **routing captures the complement**: the routed system keeps retrieval’s head F1 exactly (0.952, because almost no head merchant falls below the gate) and lifts tail F1 from 0.607 to 0.733, a 12.6 point gain, for 10.4% of transactions routed. Third, **model size matters less than evidence access, but it does matter at the margin**: routing to gpt-5-nano yields 0.670 tail F1 against 0.733 for gpt-5-mini at essentially identical cost, because cost here is dominated by the search call, not by tokens.

Figure 2 sweeps the gate threshold. The frontier is not monotone. Macro-F1 peaks at $t = 0.65$ (0.861) and *falls* to 0.848 at $t = 0.70$ and 0.790 at $t = 0.90$, while cost rises to \$0.95 per thousand. Sending more work to the model makes the system worse, because above the threshold the model is less accurate than the retrieval it overrides. The gate is not a cost dial with an accuracy penalty; it has a genuine optimum, and it is cheap.

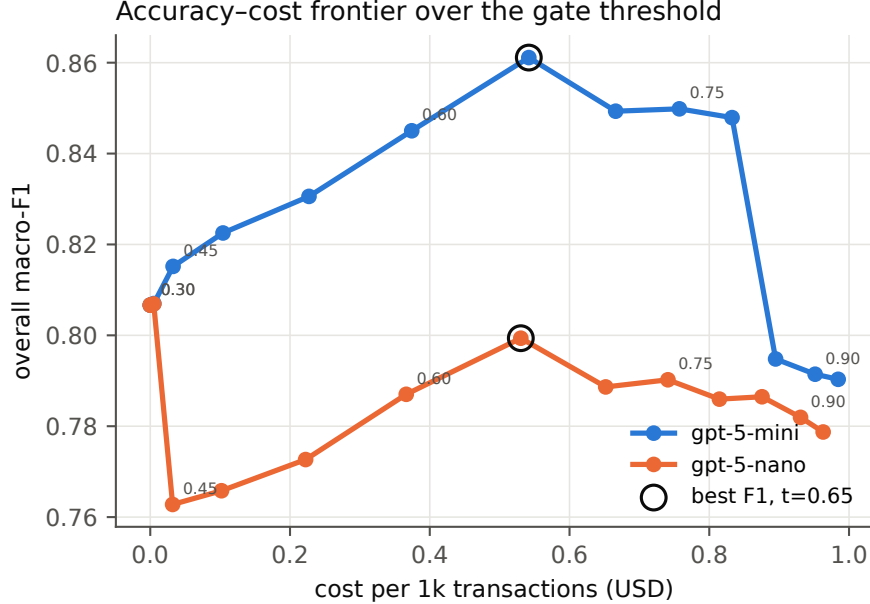


Figure 2: Accuracy-cost frontier over gate thresholds from 0.30 to 0.95. Cost is dollars per thousand transactions; the marked point is the best threshold for each model. Routing more traffic past 0.65 buys nothing: the transactions it adds are ones retrieval was already getting right.

Table 5: The critical ablation: identical prompts, merchants and taxonomy, with and without web search evidence. Paired bootstrap over merchants; every interval excludes zero.

Fallback model	Merchants	No web	With web	Difference	95% CI
gpt-5-nano	2,500	0.362	0.537	+0.175	[+0.155, +0.195]
DiffusionGemma 26B-A4B	300	0.490	0.630	+0.140	[+0.083, +0.197]
gpt-5-mini	2,500	0.501	0.624	+0.124	[+0.106, +0.141]
Muse Glimmer 30B	298	0.503	0.607	+0.104	[+0.050, +0.158]
Nemotron 3 Super 120B-A12B	319	0.511	0.577	+0.066	[+0.016, +0.116]

The three open-weight models run on NVIDIA’s free NIM endpoint and are evaluated on the shared 300-merchant tail subset, which is why their intervals are wider.

8 Is it the evidence or the model?

The central claim of any retrieval-augmented result is that the retrieved evidence, not the model, is responsible. We test it directly: same merchants, same taxonomy, same prompt template, with the web results block present or absent.

The effect is unambiguous (Table 5). Across five models spanning two orders of magnitude in size, and including three open-weight models we did not select for performance, web evidence adds between 6.6 and 17.5 accuracy points, and every paired-bootstrap interval over merchants excludes zero. The gain is *larger* for the smaller commercial model (+0.175 for gpt-5-nano against +0.124 for gpt-5-mini), which is what one expects if the evidence is supplying facts the model lacks rather than reasoning it lacks.

This also settles a cheaper hypothesis: that a large model already knows these merchants from pretraining. It does not. Without evidence, gpt-5-mini scores 0.501 on tail merchants. These are small, local, or recently founded businesses, exactly the entities least likely to be memorized.

Table 6: Cross-dataset cold start: 800 Oklahoma merchants that never appear in the DC training index, classified with the DC index and taxonomy.

Method	Macro-F1
kNN on the DC index	0.392
gpt-5-nano, no web	0.321
gpt-5-nano, web evidence	0.459
gpt-5-mini, no web	0.503
gpt-5-mini, web evidence	0.576

Table 7: Letting the model run its own searches instead of reading five fixed snippets (gpt-5-mini, the shared 300-merchant tail subset).

Evidence strategy	Accuracy	Searches/merchant	USD/1k
No web evidence	0.513	—	—
Fixed web snippets (this paper)	0.613	1.00	1.63
Agent-issued searches	0.620	1.64	21.16

Prompt sensitivity on the same subset, two independently written prompt templates: gpt-5-nano no web 1.7 pts, gpt-5-nano with web 2.7 pts, gpt-5-mini no web 2.3 pts, gpt-5-mini with web 1.3 pts. Prompt wording moves accuracy by a few points; web evidence moves it by ten to seventeen.

8.1 Cold start across institutions

Table 6 repeats the comparison on 800 Oklahoma merchants absent from the DC index, the situation a new customer or a new geography presents. Retrieval scores 0.392, essentially the guessing regime from Section 6. The web-augmented fallback scores 0.576. Note that here the no-evidence model (0.503) also beats retrieval, because retrieval has nothing to work with; the evidence still adds 7.3 points on top.

8.2 Agentic search and prompt sensitivity

Two robustness checks (Table 7). Letting the model issue its own searches through a built-in web-search tool, rather than reading five snippets we fetched, scores 0.620 against 0.613, a difference of two merchants out of 300, at 13× the cost (\$21.16 against \$1.63 per thousand). For this task the expensive agentic loop is not worth its price; one well-formed query with the merchant string is what matters. Separately, rewriting both prompts from scratch moves accuracy by 1.3 to 2.7 points, against the 10 to 17 points attributable to evidence, so the headline effect is not a prompt artifact.

9 Streaming write-back

If the fallback resolves a merchant today, the system should not pay for it again tomorrow. We replay 70,333 test transactions in arrival order through 141 windows of 500, with five write-back policies: never write; always write the resolved label; or write only when the model’s self-reported confidence exceeds a threshold.

Writing every resolved merchant back halves the bill, from \$22.62 to \$10.38 over the stream, and cuts the steady-state fallback rate from 2.99% to 1.37% (Table 8, Figure 3), at a cost of 0.3

Table 8: Streaming write-back over 70,333 test transactions in arrival order (141 windows, five seeds). Writing every resolved merchant back into the index halves the fallback bill and costs 0.3 points of macro-F1, but nearly half of the written labels are wrong.

Write-back policy	Fallback	Macro-F1	Total USD	Writes	Wrong writes	Index size
never	2.99%	0.909	22.62	0	0.0%	14,878
gated@0.8	2.99%	0.909	22.61	2	0.0%	14,880
gated@0.7	2.55%	0.905	19.26	180	22.8%	15,058
gated@0.6	2.15%	0.905	16.24	446	36.1%	15,324
gated@0.5	1.74%	0.906	13.20	709	40.6%	15,587
always	1.37%	0.906	10.38	948	46.8%	15,826

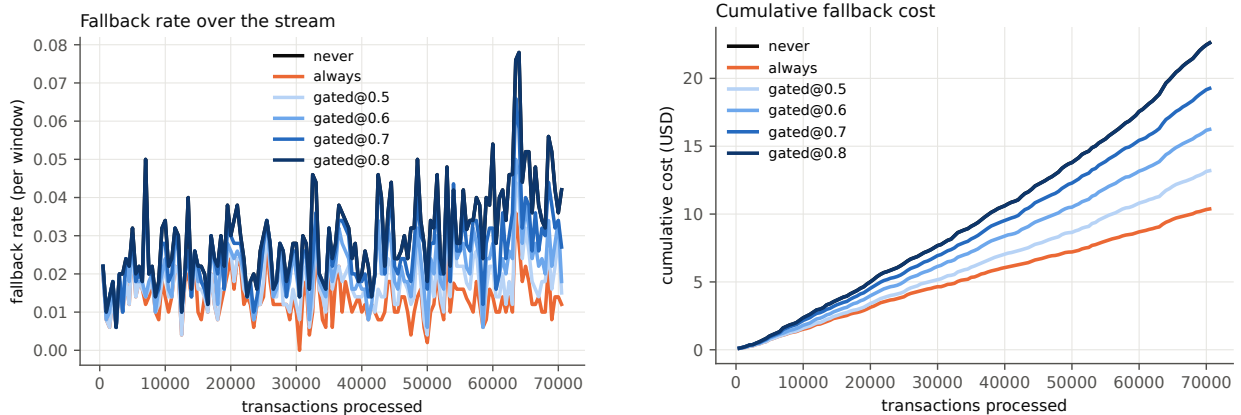


Figure 3: Left: fallback rate per window under each policy. Right: cumulative spend. Write-back bends the cost curve early and permanently, because a resolved merchant is answered by retrieval from then on.

points of macro-F1. For most deployments that is an easy trade.

The uncomfortable number is the next column: **46.8% of the written labels are wrong**. The system halves its costs while filling its own index with errors, and the aggregate metric barely notices, because the merchants being written are rare by construction and their errors are diluted. Anyone running this in production should measure the write path separately from the serving path; an aggregate F1 will not reveal the rot.

Confidence gating on the write path does not rescue it. Raising the write threshold monotonically reduces both the number of writes and their error rate, but it also removes the savings: at $t = 0.8$ the policy writes 2 merchants out of 948 candidates and is indistinguishable from never writing. These models’ self-reported confidence is not calibrated finely enough to separate right from wrong answers at any threshold that preserves the benefit [5]. A verification step with independent evidence, rather than self-reported confidence, is the obvious next design.

10 Which encoder?

Table 9 holds the index, gate, and taxonomy fixed and varies only the encoder, on the full test set. The spread is small and the ordering is boring: a commercial 1536-dimensional embedding is 1.1 points of tail macro-F1 better than a 384-dimensional CPU model at $23\times$ the encoding latency and a per-merchant API cost. Domain pretraining actively hurts: FinBERT, trained on financial

Table 9: Embedding backbones under the same index, gate and taxonomy, full DC test set, five seeds. Twenty-three times the embedding latency buys one point of tail macro-F1.

Backbone	Dim	ms/merchant	Overall	Head	Tail	Tail acc.
MiniLM-L6-v2	384	0.53	0.846	0.932	0.633	0.689
bge-small-en-v1.5	384	0.91	0.851	0.932	0.634	0.695
mpnet-base-v2	768	2.57	0.851	0.932	0.639	0.692
FinBERT	768	2.08	0.817	0.932	0.537	0.593
text-embedding-3-small	1536	11.96	0.852	0.932	0.644	0.715

Table 10: Manual taxonomy of 200 residual tail errors made by the routed system with web evidence. Two thirds are annotation or taxonomy artifacts rather than model failures.

Failure class	Count	Share	Example merchant
taxonomy mismatch	101	50.5%	WOMEN IN HOUSING & FIN
label noise	34	17.0%	MAC-ISA
LLM reasoning error	16	8.0%	BIO COMPANY INC
no web presence	15	7.5%	AUTOAUTH SERVICE
web search failure	15	7.5%	GESSCREATIONS
ambiguous merchant	13	6.5%	NATIONAL SEATING
normalization failure	6	3.0%	COURSRA2A12B1FKC1KV9Q

text, is 9.6 points of tail F1 worse than general-purpose MiniLM, presumably because merchant descriptors are names, not prose about finance. Given Section 6, none of this is surprising: no encoder can embed a merchant it has never seen into the right neighborhood, so encoder quality cannot address the dominant error mode.

11 What is actually left

We sampled 200 residual errors of the routed system on tail merchants and labeled each by hand.

Table 10 reorders the priorities. Half of the “errors” are taxonomy mismatches: the merchant is correctly identified and the predicted category is defensible, but the merchant-category-code label assigns it elsewhere. A women’s housing-finance association is coded by the acquirer as a membership organization and predicted as professional services; both are right under different readings of a 15-way taxonomy. A further 17% are outright label noise, consistent with the independent 300-row audit in Section 3. Together, two thirds of what the metric counts as failure is annotation artifact.

The genuinely addressable slice is smaller than the headline gap suggests: 8% model reasoning errors, 7.5% merchants with no usable web presence, 7.5% search failures where results existed but missed, 6.5% genuinely ambiguous merchants, and 3% normalization failures where the descriptor was mangled beyond recovery (for example a truncated course-payment reference). Only about a fifth of the remaining error is attackable by better search or a better model; the rest requires changing the label definition, which is a product decision rather than a modeling one.

12 Limitations

Domain. Both datasets are US government purchase-card programs. The merchant mix skews to office supply, industrial, and travel spending, and away from consumer categories such as groceries and restaurants. The mechanism we measure, that unseen merchants collapse and web evidence restores them, should transfer; the exact numbers should not be assumed to.

Labels. Merchant category codes are assigned by acquirers for interchange purposes, not for analytics. They are noisy at 4.4% and ambiguous at up to 19.7% (Section 3), and Section 11 shows this dominates the residual. Any comparison of systems above roughly 0.85 macro-F1 on this data is measuring annotation as much as capability.

Web drift. Search results were collected in September 2026 and frozen. The same queries tomorrow return different pages, so the absolute fallback numbers are a snapshot. This is an inherent property of web-augmented systems and an argument for caching evidence alongside predictions, which is also what makes this paper reproducible.

Open-weight coverage. The three open-weight models were evaluated on a 300-merchant tail subset because the free endpoint serializes requests; their intervals are wide and they are not directly comparable to the commercial models’ full-set numbers.

Single fallback design. We did not test multi-step verification, ensembling several models, or feeding the retrieved neighbors to the model as additional context. The write-back result in Section 9 suggests verification is the highest-value missing piece.

13 Conclusion

Embedding retrieval for transaction categorization fails in a specific, measurable way: it is near-perfect on merchants it has seen even once, and it collapses on merchants it has never seen, which are a fifth of real traffic under a temporal split. Because the failure is a missing-knowledge problem rather than a sample-size problem, the effective fix is to give a language model external evidence about the unseen merchant and to spend that model only where retrieval reports low confidence. That routed design lifts tail macro-F1 by 12.6 points for \$0.54 per thousand transactions, and the gain survives ablation across five models: it is the web evidence doing the work, not the language model. Two cautions carry beyond this dataset. Writing model answers back into the index halves cost and quietly fills the index with errors at a 47% rate, and the accuracy ceiling is set by the taxonomy and the labels rather than by the models, since two thirds of residual errors are annotation artifacts.

Reproducibility

The repository contains the pipeline, the frozen prompts with their hashes, pinned model identifiers, the committed API caches (18,928 language model responses, 3,869 searches, and float16 embedding caches for five encoders), and the taxonomy and audit files. `python reproduce.py --config configs/dc.yaml` regenerates every table and figure from those caches with no network access and verifies each regenerated table against the committed copy; a run manifest records package versions, prompt hashes, index parameters, git commit, and host. A Dockerfile builds the same environment. Table 11 reports total spend.

Table 11: Total API spend for every experiment in this paper. The frozen caches hold 18,928 model responses and 3,869 search responses; reproduce.py replays them and issues no requests.

Item	USD
OpenAI chat completions (cache pass, all experiments)	3.07
OpenAI built-in web search ablation (300 merchants)	6.38
Firecrawl search (prepaid credits, reference price)	41.44
NVIDIA NIM open-weight models (free endpoint)	0.00
Total cash spent (OpenAI)	9.45

References

- [1] Dogu Araci. FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [3] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [6] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [7] Christopher Lesner, Alexander Ran, Marko Rukonic, and Wei Wang. Large scale personalized categorization of financial transactions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [9] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [10] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- [11] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

- [12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [13] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [14] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [15] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [16] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.